

LARAVEL AFTER DEPLOY

ARCHITECTURE, PERFORMANCE,
AND OPERATIONS AT SCALE

Nuruzzaman Milon



Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

Written with	Markdown, dictated in part with Hex , an open-source, on-device voice-transcription tool, and typed the rest.
Produced with	Ibis Next , using mPDF for PDF generation and CommonMark for HTML rendering.
Trim size	7.44in x 9.69in (Crown Quarto)
Typefaces	Linux Libertine (body text), Times (headings), and 0xProto (code, SIL Open Font License 1.1).
Code width	66 characters per line, so a listing never wraps awkwardly on the page.

To my family, for making every effort worthwhile.

About the Author	670
------------------------	-----

Preface

Shipping a Laravel application is the easy part. The hard part starts after deploy: when a flash sale or a viral post turns usual traffic into a spike, when bots hammer your login endpoint, when a dependency you don't control has a bad afternoon, when the same webhook arrives twice, when the on-call phone rings at 3 a.m. and the answer isn't in the framework docs. This book is about that layer - the decisions that keep a real system fast, correct, and operable once it's already in production.

Who this book is for

This book is written for a mid-to-senior Laravel engineer who already knows how to build a Laravel application and is now being asked to keep one alive - the person a growing team hands the architecture decisions, the performance incident, and the migration that has to happen without a maintenance window. It assumes you already know how routing, Eloquent, queues, and testing work in Laravel; it doesn't stop to explain the framework. Every page is instead about the layer of decisions that sits on top of it: how you structure a codebase as more than one team starts touching it, how you keep it fast once traffic outgrows what fits comfortably on one database, how you keep it correct when the same request can arrive twice, how you keep it available when a dependency you don't control goes down, and how you run all of that at 3 a.m. on a Tuesday without it becoming a story you're still telling a year later.

If you're looking for a Laravel tutorial - how to define a route, how to write a migration, how Eloquent relationships work - this isn't that book, and there are excellent ones already. If you're the engineer who's outgrown those books and is now staring at a monorepo, a queue backlog, or a balance that doesn't add up, keep reading.

How to read it

The book is organized into eight parts, and the parts are ordered by dependency rather than importance, so reading front to back works, but it isn't the only sane way through it. Part 1 (Foundations) and Part 2 (Performance Engineering) build vocabulary that later parts assume - "shared kernel," "N+1," "backfill," and so on. Part 5 (Observability & Monitoring) is worth reading earlier than its page number suggests, because "correlation ID," "production dashboard," "SLI/SLO/SLA," and "burn rate" get reused heavily in Parts 6 and 7. Beyond that, Parts 3 through 7 are mostly skippable for a live fire - with the vocabulary caveats above - so if you picked this book up because of a live incident with your queues, your database failover, or your CI pipeline, it's reasonable to jump straight to the relevant chapter. Chapter 42, the capstone, assumes you've read - or at least skimmed - everything before it: it's a single worked incident that touches nearly every decision in the book, and it's meant to be read last.

Each chapter follows the same shape. It opens with a concrete production failure - drawn from more than fifteen years of real incidents, anonymized - then works through the pattern that would have prevented or fixed it, and closes with a short "Chapter recap" you can scan later without rereading the whole chapter. Not every one of those incidents happened in a Laravel project; over those years I've worked across many languages and frameworks, and where the original system wasn't Laravel, I've translated the failure into a Laravel setting so it can be demonstrated in the stack this book is about.

About the examples

Every incident described in this book actually happened in real production systems. The failure modes, the trade-offs, and the fixes come from systems I helped build and then had to keep running, not from thought experiments. What isn't real is any identifying detail: company names, product names, team names, exact numbers, and anything else that could point back to a specific employer or client has been changed, generalized, or recombined. The named fictional businesses that recur throughout the book - Karbar (an order-processing platform), Hishab (a wallet service), Drishti (a livestreaming and short-video app), and others - are stand-ins for this kind of company, not case studies of any single one; they exist because those are the systems that reliably produce the failure modes worth discussing, not because they map one-to-one onto any company you could name. Nothing here discloses a trade secret, a proprietary architecture, or confidential information belonging to any organization I've worked with; the anonymization exists specifically to protect that while keeping the underlying lesson intact. The architecture decisions and opinions in these pages are my own.

The code examples are written in Laravel, because that's the stack this book is built around - but most of the ideas aren't Laravel-specific. Idempotency, backpressure, observability, failure isolation, and the rest of what these chapters cover are production concerns that show up in any language and any framework. If you aren't on Laravel, the patterns still apply, even when the exact code examples don't.

id	select_type	table	type	key	rows	Extra
1	SIMPLE	products	ref	idx_category_created	1842	NULL

type: ref and a populated **key** mean the index is doing the work; **rows** drops from forty million to the actual number of matching products, and the sort disappears because the index already stores rows in the needed order.

Connection pooling basics

Every PHP-FPM worker (or queue worker process) that opens a database connection holds it for the life of the request or job. A database server has a hard ceiling on concurrent connections (**max_connections** in MySQL/PostgreSQL), and it's easy to exhaust that ceiling not through query volume but through *connection* volume: scale out enough app servers or queue workers, and the sum of their idle-but-open connections can exceed what the database allows, well before the database is doing any meaningful amount of work.

A connection pooler (PgBouncer for PostgreSQL, ProxySQL for MySQL, or a managed equivalent offered by most cloud database providers) sits between the application and the database, multiplexing many application-side connections onto a smaller pool of real backend connections. It's the standard fix once a horizontally scaled fleet outgrows what the database can hold open directly.

The math is worth doing explicitly, because it rarely feels urgent until it fails outright: twenty application servers, each running twelve PHP-FPM workers plus eight Horizon queue workers for the jobs covered in Chapter 13, is $20 \times (12 + 8) = 400$ potential concurrent connections from a single application - before a second app in the monorepo (Chapter 3) opens its own connections

to the same database, and before a rolling deploy briefly runs old and new workers side by side. A `max_connections` setting of 500 has far less headroom than it looks like on paper.

Approach	What it solves	What it doesn't solve
Increase <code>max_connections</code>	Buys headroom short-term	Increases memory pressure per connection; doesn't fix the underlying growth curve
Connection pooler (PgBouncer/ProxySQL)	Decouples app-side connection count from real backend connections	Adds an operational component and a small latency hop
Persistent connections (PDO)	Reduces per-request connect overhead on a single host	Doesn't reduce the <i>number</i> of held connections across a fleet - can make pooling more urgent, not less

Connection pooling is a fleet-level concern, not a per-app config flag - it belongs in the same conversation as horizontal scaling and read replicas (Chapter 25 covers the high-availability side of the data layer in more depth).

Pagination at scale: normal, simple, and cursor

Karbar's admin panel has a searchable orders table. It works fine at ten thousand rows. At two million rows, the page takes eight seconds to load - and profiling shows the slow part isn't the twenty-five rows being displayed, it's the `COUNT(*)` query Laravel's default paginator runs to compute the total page count for the "Page 3 of 84,213" control.

Pagination type	Runs a <code>COUNT(*)</code> ?	Deep-page cost	Supports "jump to page N"?	Best for
<code>paginate()</code>	Yes, every request	Degrades with table size (count + offset scan)	Yes	Small-to-medium tables, UI needs page numbers
<code>simplePaginate()</code>	No	Still degrades with offset depth, but skips the count cost	Next/previous only	Medium tables where total count isn't needed
<code>cursorPaginate()</code>	No	Flat - constant cost regardless of depth, given an index	Next/previous only	Large tables, infinite-scroll or API pagination

Cursor pagination avoids `OFFSET` entirely by encoding the last-seen row's sort key into an opaque cursor and asking for everything that sorts after it - a query an index serves directly, regardless of how deep into the table the user has scrolled:

```
$orders = Order::query()
    →orderByDesc('created_at')
    →orderByDesc('id')
    →cursorPaginate(25);
```

Under the hood, that opaque cursor is just a base64-encoded tuple of the last row's sort columns, and it turns the next page into a comparison the index can seek to immediately, instead of a **LIMIT ... OFFSET** the database has to count past:

```
-- What paginate()/simplePaginate() run for page 84,213 (offset
2,105,300):
SELECT * FROM orders ORDER BY created_at DESC, id DESC LIMIT 25
OFFSET 2105300;

-- What cursorPaginate() runs for the equivalent next page - a
seek, not a scan.
-- Laravel spells the tuple comparison out longhand rather than
using row values:
SELECT * FROM orders
WHERE (created_at < '2026-03-14 09:12:00'
      OR (created_at = '2026-03-14 09:12:00' AND id < 84213))
ORDER BY created_at DESC, id DESC
LIMIT 25;
```

The offset version still has to walk (and discard) 2,105,300 rows before it can return the 25 that matter, even though the database never sends them to the application. The cursor version seeks directly to a known point via the **(created_at, id)** index and reads only the 25 rows it returns - which is why its cost stays flat whether the cursor points at row 25 or row 2,500,000.

The tradeoff is real: cursor pagination can't render "Page 3 of 84,213" or let a user jump straight to page 50, because there's no cheap way to know how many rows precede an arbitrary offset without counting them. For the admin panel, the fix was to drop the total-count display and page-number control in favor of next/previous cursor navigation - a small UX change that turned an eight-second page load back into a two-hundred-millisecond one.

Chapter recap

- Dev-scale seed data hides exactly the problems that matter at production scale - test query performance against realistically sized datasets, not two hundred seeded rows.
- Eager load relations (**with**, **loadMissing**) and project only needed columns (**select**) to eliminate both query-count and I/O blowups in loops; **Model :: preventLazyLoading()** turns a missing eager load into a failing test instead of a slow endpoint three months later.
- Index the columns your queries actually filter and sort by, in the order they're used, and don't expect a low-selectivity column to earn its own index.
- **EXPLAIN (type, key, rows, Extra)** turns "this feels slow" into a specific, fixable finding - a full scan, a missing key, or an in-memory sort.
- Connection pooling (PgBouncer/ProxySQL) is what keeps a horizontally scaled fleet from exhausting the database's connection ceiling, independent of query volume.
- Offset-based pagination's **COUNT(*)** and deep-offset scans both degrade as a table grows; cursor pagination stays flat at the cost of losing total counts and arbitrary page jumps - a fair trade for large tables.

Chapter 9 - Safe Schema Evolution at Scale

The migration that only failed in production

At Karbar, a fictional order-processing platform, a routine cleanup task renames a column: `orders.buyer_email` becomes `orders.customer_email`, to match naming used everywhere else in the domain. The migration is small, obviously correct, and reviewed without objection. In staging, against a ten-thousand-row table, it runs in forty milliseconds. In production, against a table with two hundred million rows, the `ALTER TABLE` takes an exclusive lock for twenty-five minutes. Every write to `orders` blocks behind it. The API starts returning timeouts, the on-call engineer gets paged, and the migration that "obviously" worked has to be killed mid-run, leaving the schema in an ambiguous state.

Nothing about the migration was wrong, syntactically. It was tested at the wrong scale, the same failure mode as Chapter 8, applied to schema changes instead of queries. This chapter covers the patterns that make schema changes safe regardless of table size: non-blocking DDL, a backward-compatible rename pattern, batched backfills, and - the part teams forget most often - tracking who else reads your schema before you assume a "safe" change is actually safe for everyone.

Why "fast in staging" doesn't transfer

The cost of a schema change is a function of data volume, not query complexity, and that cost varies by database engine and by the specific

operation:

- Adding a nullable column with no default is metadata-only and near-instant on both MySQL (InnoDB, modern versions) and PostgreSQL, regardless of table size.
- Adding a column with a default value, or adding a **NOT NULL** constraint to an existing column, historically required rewriting every row to populate or validate the new value - a cost proportional to table size. Recent versions of both engines have narrowed this (PostgreSQL avoids a full rewrite for constant defaults since version 11; MySQL's behavior depends on the specific **ALGORITHM** used), but the safe assumption is: verify the specific operation against the specific engine version in use, on a copy of production-sized data, before trusting it in staging.
- Renaming a column, dropping a column, or changing a column's type can each range from instant metadata changes to full table rewrites, again depending on engine and version.

The rule that survives all of that engine-specific detail: never trust a migration's timing from a small staging database. Run it against a production-sized copy (a recent snapshot, restored somewhere disposable) before it ships, and know in advance whether the specific operation takes a blocking lock.

Concretely, adding a **NOT NULL** constraint to **orders.customer_email** on PostgreSQL as a single step forces the database to scan and validate every one of two hundred million rows while holding a lock that blocks writes for the duration:

```
/*  
 * Locks orders for the full validation scan - the whole table,  
 * all at once.  
 */  
Schema::table('orders', function (Blueprint $table) {  
    $table->string('customer_email')->nullable(false)->change();  
});
```

Splitting it into "add the constraint without enforcing it yet," "validate it separately," and only then "make the column formally **NOT NULL**" turns one long exclusive lock into a near-instant metadata change plus a scan that only takes a brief lock at the very end:

```
-- Step 1: instant - records the constraint without checking
existing rows.
ALTER TABLE orders ADD CONSTRAINT customer_email_not_null
    CHECK (customer_email IS NOT NULL) NOT VALID;

-- Step 2: scans the table, but only takes a brief lock to flip
the
-- constraint's state once the scan finds no violations - not for
the
-- duration of the scan itself.
ALTER TABLE orders VALIDATE CONSTRAINT customer_email_not_null;

-- Step 3: after the validated check proves there are no nulls,
PostgreSQL
-- can set the column attribute without repeating the full-table
scan.
ALTER TABLE orders
    ALTER COLUMN customer_email SET NOT NULL;
```

Run as separate deploys, the first one is safe to ship immediately; validation and the final **SET NOT NULL** can run once the backfill from the next section has finished, with no code depending on their timing.

Concurrent and non-blocking index creation

Adding an index to a large table is one of the most common migrations to get wrong, because a plain **CREATE INDEX** takes a lock that blocks writes for as long as the index build takes - which, on a two-hundred-million-row table, can be tens of minutes.

PostgreSQL's answer is **CREATE INDEX CONCURRENTLY**, which builds the

index without holding a write lock, at the cost of a slower two-pass build and a real failure mode: if it's interrupted, it can leave behind an invalid index that has to be dropped and retried, not resumed.

```
CREATE INDEX CONCURRENTLY idx_orders_customer_created
ON orders (customer_id, created_at);
```

MySQL's InnoDB engine supports online DDL for many index operations via `ALGORITHM=INPLACE, LOCK=NONE`, which allows concurrent reads and writes during the build - but not every DDL operation supports every algorithm, and the safe habit is to check the specific operation against the running engine version rather than assume.

```
ALTER TABLE orders
ADD INDEX idx_orders_customer_created (customer_id,
created_at),
ALGORITHM=INPLACE, LOCK=NONE;
```

Laravel's migration files don't manage this distinction automatically - a `Schema::table()` migration using `$table->index()` issues a plain `CREATE INDEX/ADD INDEX` unless the raw SQL or a database-specific driver option is used to request the non-blocking variant. On a large table, that's a decision to make explicitly, not a default to trust:

```
public function up(): void
{
    /*
     * Schema::table()->index() takes a blocking lock for the
     * build - fine on a small table, a production incident on a
     * two-hundred-million-row one.
     */
    DB::statement(
        'CREATE INDEX CONCURRENTLY idx_orders_customer_created'
        .' ON orders (customer_id, created_at)'
    );
}
```

CREATE INDEX CONCURRENTLY can't run inside the transaction Laravel wraps each migration in by default, so the migration class also needs **public \$withinTransaction = false;** - forgetting that is the most common way this pattern fails silently in a migration file, even though it works fine when run by hand. And because an interrupted concurrent build leaves an unusable index behind rather than rolling back, always check for one before retrying:

```
-- Finds indexes left in an invalid state by an interrupted
CONCURRENTLY build.
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE indisvalid = false;

-- Safe to drop and retry - an invalid index is never used by the
planner.
DROP INDEX CONCURRENTLY idx_orders_customer_created;
```

The backward-compatible column rename pattern

Renaming a column directly - `ALTER TABLE orders RENAME COLUMN buyer_email TO customer_email` - is fast as a pure metadata operation on most engines. The real danger isn't the `ALTER` itself; it's that the moment it commits, every piece of code still referencing the old column name breaks atomically, everywhere, at once - including code you don't control (see the next section). The fix is to never do a rename as a single step. Expand it into phases that can each be shipped, verified, and rolled back independently:

Phase	Schema state	Application state	Rollback if something's wrong
1. Add	New column exists, nullable	Not yet used	Drop the new column - no impact

Phase	Schema state	Application state	Rollback if something's wrong
2. Dual-write	Both columns exist	App writes to both columns on every write path; reads still from the old column	Stop dual-writing - old column is still authoritative
3. Backfill	Both columns exist	Batch job populates the new column for existing rows	Backfill is idempotent - safe to re-run or pause
4. Migrate reads	Both columns exist	App reads from the new column; old column no longer read	Revert the read change - old column is still fully populated
5. Stop dual-write	Both columns exist	App writes only to the new column	Revert to dual-writing if any consumer still expects the old column
6. Drop	Old column removed	Fully migrated	Not reversible past this point - this phase only ships once nothing reads the old column

Each phase is a separate, independently deployable change. If phase 4 reveals a bug (the new column has a subtly different meaning than assumed), the rollback is "revert one deploy," not "recover from a completed rename." This is the same expand-and-contract shape used for the shared-package upgrades in Chapter 7 - the pattern generalizes to any change where "old" and "new" need to coexist for a transition window.

Phase 1 is the only phase that touches the database schema directly - it's a plain, additive migration:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        // after() is MySQL-only; omit it on PostgreSQL.
        $table->string(
            'customer_email'
        )->nullable()->after('buyer_email');
    });
}
```

Phase 2's dual-write lives in the model, not in a migration, and is the one line of application code every write path now depends on until phase 5 removes it - with the caveat that "every write path" means every write path that goes through a model instance. `Order::query()->update( ... )` and anything built on `DB::table('orders')` skip model events entirely, so grep for those before trusting this hook, because phase 4's correctness rests on it:


```
protected static function booted(): void
{
    static::saving(function (Order $order) {
        if ($order->isDirty('buyer_email')) {
            $order->customer_email = $order->buyer_email;
        }
    });
}
```

Phase 6's drop is deliberately its own migration, shipped only after the checklist at the end of this chapter is fully checked off:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        $table->dropColumn('buyer_email');
    });
}
```

Batching large backfills

Phase 3 above - populating the new column for two hundred million existing rows - is itself a hazard if done as one statement:

```
// Don't do this on a large table.
DB::table('orders')
    ->update(['customer_email' => DB::raw('buyer_email')]);
```

A single **UPDATE** touching every row holds row locks (and, depending on

engine and isolation level, can hold them for the duration of one very long transaction), competes with live traffic for the same rows, and - on replicated setups - can create significant replication lag, since a replica applies the same enormous write as one unit. The safe alternative is chunking: touch a bounded number of rows per statement, with a small pause between batches to let replicas catch up and to leave room for live traffic.

```
Order :: query()
  →whereNull('customer_email')
  →orderBy('id')
  →chunkById(1000, function ($orders) {
    Order :: query()
      →whereIn('id', $orders→pluck('id'))
      →update(
        ['customer_email' => DB::raw('buyer_email')]
      );

    // 100ms - bound replication lag and lock contention
    usleep(100_000);
  });
```

Running this as a resumable Artisan command that processes one bounded batch (tracking the last processed ID) rather than a single migration step means it can be paused, monitored, and restarted without redoing completed work - important for a backfill that might run for hours against a genuinely large table:

```
final class BackfillCustomerEmail extends Command
{
    protected $signature =
        'orders:backfill-customer-email'
```

```

        .' {--fresh} {--dry-run} {--batch=1000}';

public function handle(): int
{
    $lastId = $this->option(
        'fresh'
    ) ? 0 : (int) DB::table('backfill_progress')
        ->where('name', 'customer_email')
        ->value('last_id');

    $dryRun = $this->option('dry-run');
    $orders = Order::query()
        ->whereNull('customer_email')
        ->where('id', '>', $lastId)
        ->orderBy('id')
        ->limit((int) $this->option('batch'))
        ->get(['id']);

    if ($orders->isEmpty()) {
        $this->components->info(
            'Backfill complete: no null emails left.'
        );

        return self::SUCCESS;
    }

    $nextLastId = $orders->last()->id;

    if (! $dryRun) {
        /*
         * whereNull again, so a concurrent write that
already
         * populated one of these rows isn't overwritten by a
         * stale read.
        */
    }
}

```

```

    Order::query()
      →whereIn('id', $orders→pluck('id'))
      →whereNull('customer_email')
      →update(
        ['customer_email' => DB::raw('buyer_email')]
      );

    DB::table('backfill_progress')→updateOrInsert(
      ['name' => 'customer_email'],
      ['last_id' => $nextLastId],
    );
  }

  $this→components→info(
    ($dryRun ? '[dry run] ' : '')
    ."Processed {$orders→count()} rows"
    ." (last ID: {$nextLastId})."
  );

  return self::SUCCESS;
}
}

```

Dispatched via `Schedule::command('orders:backfill-customer-email')→everyMinute()→withoutOverlapping()`, this makes progress in small, bounded increments and survives being interrupted by a deploy at any point. The cursor lives in a table rather than in the cache for a boring reason that bites people anyway: a cache is allowed to lose your key. An eviction under `maxmemory`, a `cache:clear` during a release, a Redis restart without persistence - any of those silently resets the backfill to ID 0, and the `whereNull` guard means it will quietly grind back through two hundred million already-completed rows without ever reporting that anything went wrong.

`--dry-run` is worth adding to every backfill command before it ever touches a large table: it walks the exact same query and batch-selection logic, reports how many rows and which ID range the next batch would affect, and writes nothing - so a mistake in the `WHERE` clause shows up as a suspicious row count in dry-run output, not as two hundred million incorrectly updated rows. And it's always worth emitting a progress line per batch, not just a single message at the end - a backfill that runs for hours with no output is indistinguishable from one that's silently hung, and the per-batch line (row count, last ID reached) is what turns "is this still running?" into a question the terminal already answers.

Downstream consumers: your schema is a public API

The phased rename above protects the application. It does nothing for anything else that reads the same database directly and isn't part of the deploy you control: a nightly analytics export, another internal service with direct read access to the same tables (an anti-pattern, but a common inherited one), or a BI tool querying a read replica. From their perspective, dropping `buyer_email` in phase 6 is a breaking change shipped with zero notice, because nothing about the application's own migration process is aware they exist.

The fix starts with an inventory: before any schema change on a table of nontrivial age, identify who reads it besides the application - export jobs, other services, reporting queries, scheduled dashboards. For Karbar's rename, that inventory turned up an internal reporting export that ran nightly against `orders.buyer_email` directly, owned by a different team on a different release cycle.

Two patterns handle this without forcing every downstream consumer onto the application's timeline:

```

redis.call('SET', KEYS[1], 0)
return taps
LUA,
    1,
    "stream:{$this→streamId}:reactions",
);

if ($count > 0) {
    broadcast(
        new ReactionCountUpdated($this→streamId, $count)
    );
}

/*
 * Re-queue for a second from now instead of broadcasting
 * per tap, as long as the stream is still live.
 */
if (Stream::find($this→streamId)?→is_live) {
    self::dispatch($this→streamId)
        →delay(now()→addSecond());
}
}
}

```

The tap handler itself never broadcasts anything, it just does `Redis::incr("stream:{$streamId}:reactions")` and returns. The Lua script is doing real work here, and it's worth being clear about why `Cache::pull()` isn't a substitute for it: `pull()` is a `get()` followed by a `forget()`, two separate round trips, and every tap that lands in between them is read by nobody and then deleted. On a channel quiet enough to be worth coalescing you'd never notice; on the viral one this whole section exists for, you'd be dropping reactions at exactly the moment people are watching the counter.

- **Drop, don't queue, for freshness-only data.** If a client is behind on viewer-count updates, the right behavior is usually to deliver the latest value and discard the stale ones in between, not to queue every intermediate value for eventual delivery.
- **Cap per-connection buffer size** at the connection-layer configuration level, so one slow client can't accumulate unbounded backlog and start consuming memory disproportionate to every other connection.

The general principle: not every real-time event needs guaranteed, ordered, lossless delivery. Deciding which events tolerate being coalesced or dropped, versus which ones must never be lost, is a design decision that has to happen before the burst, not during the incident.

Reconnect and replay: not silently missing events

WebSocket connections drop - a mobile client loses signal, a load balancer cycles connections, a deploy restarts the connection layer. A client that just silently reconnects and resumes listening has a gap: whatever was broadcast during the disconnect window is gone, and unless the application is deliberate about it, the client has no idea anything was missed.

Two complementary patterns close that gap:

- **Sequence or timestamp markers on broadcast payloads**, so a reconnecting client can ask "give me everything since sequence N" as a normal HTTP request against the origin data, rather than relying on the WebSocket layer to have retained anything.
- **A subscribe-then-reconcile handshake on every reconnect**, not just on the very first load. Treat "the socket just reconnected" as equivalent to "the client's local state might be stale." Fetching history

before resubscribing recreates a gap for events emitted during the fetch; subscribe first, buffer live events, reconcile the authoritative delta, then merge.

The client-side handshake, in order:

1. Echo detects the disconnect and reconnects with backoff - that part you get for free.
2. On reconnect, resubscribe immediately and buffer the incoming broadcasts by sequence, without applying them to UI state yet.
3. Fetch the authoritative gap: **GET**
/api/streams/{id}/chat?since=<last_seen_sequence>.
4. Apply the fetched delta, then merge the buffered events, deduplicating by sequence so an event that arrived both live and in the fetch is applied once.

Skipping this handshake - or doing the fetch before the subscribe - is what turns "the connection blipped for four seconds" into "half the users in this channel silently missed three chat messages and don't know it" - invisible in a demo, and one of the most common sources of "the feature feels randomly unreliable" reports in a production real-time system.

Worked example: finding the bottleneck under load

When a real-time feature degrades at scale, the diagnostic order matters, because the three plausible culprits (origin event volume, broadcaster fan-out, connection-layer capacity) fail in different, distinguishable ways:

Symptom	Likely bottleneck	How to confirm
App servers and database show normal load; events still fire, but clients stop receiving updates	Connection layer at capacity	Check open-connection count per node against configured limits; look for connection-layer CPU/memory saturation, not app-tier metrics
Broadcaster publish latency or Reverb inter-node delivery latency grows while app and connection nodes look healthy	Fan-out mechanism between app and connection nodes	Measure Redis publish latency and publish-to-delivery time; Redis pub/sub has no durable backlog to inspect
Broadcast queue's oldest-job age grows while Reverb publish latency stays flat	Broadcast workers or a competing default queue are saturated	Inspect the broadcast queue separately; move it to a dedicated queue or add workers before scaling the connection layer
App servers show high load before jobs are enqueued	Origin event volume, not real-time delivery	Check whether events are being generated faster than intended and whether coalescing should happen before dispatch

For Drishti, the first table row was the answer: connection-layer nodes were pegged at their configured connection ceiling, the app and database were idle, and the fix was horizontal scaling of the connection tier plus a Redis pub/sub

backbone to coordinate fan-out across the now-multiple nodes, alongside coalescing the highest-frequency event (reaction taps) so growth in viewer count didn't translate one-to-one into growth in messages-per-second on the busiest channel.

Chapter recap

- A **ShouldBroadcast** event passes through five links: event, queued broadcast job, broadcaster, connection layer, and client. Queue lag is part of real-time latency; **ShouldBroadcastNow** deliberately moves that work inline.
- The cost that actually matters is fan-out: events per second on a channel times subscribers on that channel, not the raw event rate your application produces.
- Presence channels are the most expensive channel type because every join and leave is itself broadcast to every other member - don't use one just to display a count.
- The connection layer scales on connections-per-node, not requests-per-second, and needs its own capacity planning, deployment, and monitoring, separate from the HTTP app tier.
- Handle bursts with backpressure: coalesce noisy events before broadcasting, drop stale freshness-only data instead of queuing it, and cap per-connection buffers so one slow client can't exhaust shared memory.
- Reconnects need an explicit reconciliation fetch against authoritative origin data, not just a resumed subscription - otherwise clients silently miss whatever was broadcast during the gap.

Chapter 16 - Rate Limiting & Abuse Protection

2 a.m., and the login endpoint won't stop ringing

The graphs look like an attack, because it is one. Login attempts on Adda, a fictional social platform, have gone from a baseline of a few hundred per minute to over forty thousand, spread across tens of thousands of IP addresses, each one trying a handful of username/password combinations before moving on. It's credential stuffing: a botnet replaying a leaked password list from an unrelated breach, hoping a slice of users reused the same password here.

The naive fix - block anyone who fails login more than five times - makes things worse. The attack is already distributed across so many source IPs that per-IP blocking barely slows it down, while a support queue fills with real users who mistyped a password twice and are now locked out for an hour. The team that survives this incident well isn't the one with the cleverest CAPTCHA; it's the one that had already decided, in calmer times, what a rate limit protects, at what granularity, and what happens to a legitimate request caught in the blast radius.

That's the actual subject of this chapter: rate limiting is not one mechanism, it's a layered set of decisions about *who* you're limiting, *what unit of time* you're limiting them over, and *what you do* when the limit is hit - decisions that are much cheaper to make before an attack than during one.

Two limiter algorithms, and why the difference matters

Laravel ships with one of the two algorithms below, and it's worth knowing which one before you assume an endpoint is protected: the `RateLimiter` facade and the `throttle` middleware both count requests in cache-based fixed windows.

Fixed window counts requests in discrete buckets (e.g. "requests this calendar minute"). It's cheap - one counter, one TTL - but it has a boundary problem: a client can send its full limit at 11:59:59 and its full limit again at 12:00:01, getting two limits' worth of traffic in two seconds.

Token bucket (and its close cousin, the sliding-window log) models a bucket that refills at a steady rate and is drained by each request. It smooths out the boundary problem because there's no reset instant - the bucket's state is continuous. It costs slightly more to compute (you need the last-refill timestamp, not just a count) but it's a much closer match to "how many requests can this client sustainably send," which is usually the thing you actually care about.

	Fixed window	Token bucket / sliding window
Storage cost	One counter + TTL per key	Timestamp + count per key
Boundary burst	Up to 2x limit across a window edge	Bounded by bucket size, not window edge

	Fixed window	Token bucket / sliding window
Good for	Coarse, cheap limits (e.g. "1000 req/hour per API key")	Limits where burst behavior matters (login, payment attempts)
Laravel primitive	<code>RateLimiter::attempt()</code> with default cache-based windows	Custom atomic bucket in Redis or an edge-provider limiter

For most application-level throttles, the fixed window's imprecision doesn't matter - nobody cares if a reporting endpoint allows a small burst across a minute boundary. For anything security-sensitive (login, password reset, payment attempts, OTP verification), the boundary burst is exactly the gap an attacker will find, so it's worth the extra complexity of a token bucket - which you will be building yourself, in Redis or at the edge, because it isn't a mode you can switch the stock limiter into.

Choosing the key: per-user, per-IP, per-action, or a combination

A rate limit is only as good as the key it's keyed on, and this is where most naive implementations fail.

- **Per-IP** limits stop a single machine from hammering an endpoint, but are close to useless against a distributed botnet, and actively harmful behind carrier-grade NAT or corporate proxies, where thousands of

real users share one visible IP.

- **Per-user (or per-account)** limits are precise but only apply once you know who the user is - they don't help on an unauthenticated login attempt where the "user" is exactly what's being guessed.
- **Per-action** limits (e.g. "5 password reset emails per hour, regardless of who's asking") protect a shared downstream resource (an email provider, an SMS gateway) from being exhausted by any single actor.
- **Composite keys** - IP *and* the attempted username, or device fingerprint *and* account - are usually where the real protection lives, because they let you set a tight limit on the narrow combination ("this IP trying this specific username") without punishing an entire shared IP range for one bad actor.

The credential-stuffing scenario above is precisely why single-axis keys fail: per-IP limiting is defeated by the botnet's IP diversity, and per-username limiting alone would let the botnet grind through the account list one attempt per username, staying under any reasonable per-account threshold. The fix is layered limits on different keys simultaneously, each cheap on its own, expensive to defeat all at once.

```
/*
 * A layered login throttle: IP-wide, then IP+username, then
 * account-wide.
 */
public function attempt(Request $request): RedirectResponse
{
    $ip = $request->ip();
    $username = (string) $request->input('email');

    $limits = [
        // this IP, any account, per minute
        "login-ip:{$ip}" => 30,
```

```
// this IP against this account
"login-ip-user:{$ip}:{$username}" => 5,
// this account, from anywhere
"login-user:{$username}" => 10,
];

foreach ($limits as $key => $maxAttempts) {
    if (RateLimiter::tooManyAttempts($key, $maxAttempts)) {
        $seconds = RateLimiter::availableIn($key);

        throw ValidationException::withMessages([
            'email' =>
                "Too many attempts. Retry in {$seconds}s.",
        ]->status(429);
    }
}

if (! Auth::attempt($request->only('email', 'password')) {
    foreach (array_keys($limits) as $key) {
        /*
         * The first failed attempt must create the fixed
         * window's TTL.
         */
        RateLimiter::hit($key, decaySeconds: 60);
    }

    throw ValidationException::withMessages([
        'email' =>
            'These credentials do not match our records.',
    ]);
}

/*
 * Successful logins do not increment any of the failure
 * counters.
 */
```

```
*/  
return redirect()->intended('/dashboard');  
}
```

Hitting the limiter only on *failed* attempts, rather than on every request, is what keeps this from punishing legitimate traffic. A limiter that counts successes too will eventually lock out the user whose only crime is logging in a lot.

Bot and risk scoring: don't let a vendor outage become your outage

Many teams layer a third-party risk-scoring or bot-detection service (device fingerprinting, behavioral scoring, a managed CAPTCHA) on top of application-level rate limits. These are valuable, but they introduce a dependency that can fail independently of your own infrastructure - and a naive integration turns "the risk vendor is slow" into "nobody can log in."

The rule that matters here: **decide, explicitly, what happens when the risk check itself fails or times out**, rather than letting an exception bubble up and 500 the request. There are two defensible postures, and the right one depends on what the endpoint protects:

Failure mode	Fail closed (block)	Fail open (allow)
Behavior on vendor timeout	Treat as high-risk, block or challenge	Treat as unscored, let the request through

Failure mode	Fail closed (block)	Fail open (allow)
Right for	High-value, low-volume actions (large withdrawals, account recovery)	High-volume, low-marginal-risk actions (viewing a page, browsing a catalog)
Failure mode if wrong	Vendor blip becomes a full outage for real users	A vendor outage becomes a temporary abuse window
Mitigation	Short timeout (100-300ms) + circuit breaker so a slow vendor degrades to "unscored" quickly	Alert loudly on sustained fail-open so someone investigates rather than it becoming permanent

For Adda's login endpoint, the pragmatic default is: apply the application-level rate limits (which are self-hosted and always available) unconditionally, and treat the risk score as an *additional* signal that can tighten the response (e.g. require a CAPTCHA) but should not be a hard dependency for the base throttle to function. That way a risk-vendor outage degrades the system to "slightly less sophisticated bot detection," not "login is down."

In code, that means a tight timeout on the vendor call and an explicit fail-open branch, rather than letting the exception propagate:

```
public function scoreLogin(string $ip, string $email): ?RiskScore
{
    try {
        $response = $this->http
```

```
        // 250ms: fail fast, don't hold up the login request
        ->timeout(0.25)
        ->post('https://risk-vendor.example.com/v1/score', [
            'ip' => $ip,
            'email' => $email,
        ]);

    if ($response->failed()) {
        /*
         * A 4xx/5xx arrives here, not in the catch block
         * below.
         */
        Log::critical(
            'Risk vendor returned an unsuccessful response',
            [
                'status' => $response->status(),
            ]
        );

        return null;
    }

    return RiskScore::fromArray($response->json());
} catch (ConnectionException $e) {
    /*
     * Fail open: an unscored login still passes the
     * application-level throttle above, it only skips the
     * extra CAPTCHA/challenge step.
     */
    Log::critical(
        'Risk vendor unavailable, failing open',
        ['exception' => $e->getMessage()]
    );

    return null; // caller treats null as "unscored"
}
```

```

        | fails: no slot available
        v
+-----+
| Refund |
| charge |
+-----+
        |
        v
+-----+
| Release|
| inventory|
+-----+

```

Nothing above rolls back a shared transaction - there isn't one. Each compensating action is its own independent, idempotent write that undoes the *business* effect of a step that already committed.

```

final class PlaceOrderSaga
{
    public function run(Cart $cart): Order
    {
        $reservation = $this->inventory->reserve($cart);

        try {
            $charge = $this->gateway->charge(
                $cart->totalCents(),
                $cart->paymentMethodId
            );
        } catch (ChargeFailedException $e) {
            $this->inventory->release($reservation);
            throw $e;
        }
    }
}

```

```
try {
    $slot = $this->fulfillment->allocateSlot(
        $cart,
        $reservation
    );
} catch (NoSlotAvailableException $e) {
    $this->gateway->refund($charge->id);
    $this->inventory->release($reservation);
    throw $e;
}

return $this->orders->confirm(
    $cart,
    $reservation,
    $charge,
    $slot
);
}
```

Two things about this code matter more than the happy path. First, every compensating action (**release**, **refund**) has to be idempotent and safe to call on a step that may have already been rolled back by a previous retry - the saga itself can crash mid-compensation, and whatever resumes it will call the same compensating actions again. Second, a saga's compensations are not a true undo: refunding a charge does not un-happen the charge from the customer's point of view (it's a second event, a refund, that a bank statement will show separately), and releasing an inventory reservation does not guarantee the same units are still available when someone else wants them. Sagas manage business-level correctness, not the illusion of the operation never having been attempted.

For flows longer than a couple of steps, encode the saga as an explicit state machine - persisted in a table with columns like **step**, **status**, and enough context to resume or compensate from any point - rather than as in-memory sequential code in one request. A queue worker can then pick up a saga that stalled mid-flow (because a deploy happened, because a process died) and resume exactly where it left off, instead of the whole flow being lost the moment the original request's process exits.

	Outbox pattern	Saga
Best for	One write plus one or more downstream side effects	Multiple ordered steps across several systems
Atomicity guarantee	The local write and the event are atomic; the side effect is "at least once, eventually"	None of the steps are atomic with each other; consistency comes from compensations
Failure handling	Retry the event until the side effect succeeds	Run compensating actions for whatever already succeeded
Needs a persisted state machine?	Not usually - one event, one or more independent consumers	Usually, for anything beyond two or three steps
What it can't undo	Nothing - it never claimed the first write "didn't happen"	The real-world effect of prior steps (a charge, a reservation) - only its logical accounting

The external system is the source of truth for "did it happen"

Whatever pattern is in place, application state can still drift from reality: a webhook gets dropped, a queue job that was supposed to process an `order_events` row dies without retrying, a network partition delays a charge confirmation past your retry window. The only durable fix is to never let your own database be the last word on "did the gateway actually charge this card." Periodically (a scheduled job, on the order of minutes for anything money-related) reconcile against the gateway's own record of transactions - list charges from the last N hours, compare against local `orders/order_events`, and treat any mismatch as an incident to resolve rather than a rare glitch to ignore.

This has a direct implication for how idempotency keys are chosen (Chapter 19): they need to be stable across retries of the *same logical attempt* originating from your application, so that when reconciliation finds a gateway charge with no matching local order, replaying the intent is safe - it either finds the existing order or creates exactly one, never two.

Designing the UI for eventual consistency

Once "recorded" and "confirmed" can legitimately be milliseconds or minutes apart, the UI has to stop assuming they're the same moment. A checkout confirmation page that says "Payment successful" the instant the gateway responds, before the internal order-processing saga has even started, is lying about how much has actually happened - and if a later saga step fails, that page's promise is now wrong. The safer pattern is an explicit `processing` state that the UI shows honestly ("Payment received, confirming your order...") backed by a status endpoint or a broadcast event that flips the page

to **confirmed** or **failed** once the saga resolves, rather than a boolean that's true the moment the first system says yes.

Chapter recap

- A dual write across your database and an external system (a payment gateway, another service) cannot be made atomic - the two commits are inherently independent, and the failure gap between them is the actual problem this chapter solves.
- Two-phase commit isn't available for most real dependencies, because the dependency's API doesn't expose a "prepare, then commit or abort on command" step - build for eventual consistency instead of pretending atomicity is achievable.
- The outbox pattern makes the *intent* atomic: record the outcome and a durable event in the same local transaction, and drive every downstream side effect off that event instead of off code that may never run if the process dies.
- An outbox relay claims committed rows with a short lease, publishes with a stable idempotency key, and marks success afterward. A crash between publish and mark creates duplicate delivery by design, so consumers need inbox-style deduplication and poison events need an alerted dead-letter state.
- Sagas coordinate multi-step flows across systems using compensating actions for whatever already succeeded - persist saga state explicitly once a flow has more than a couple of steps, so a stalled saga can be resumed instead of lost.
- Treat the external system as the source of truth for "did it really happen," and reconcile against it on a schedule - application state will drift eventually, and the fix is detection, not prevention.
- Design the UI to represent "processing" as its own honest state, not as a premature "success" that a later failure would make untrue.

Chapter 23 - Working with Datetime and Timezones

The streak that broke at midnight

Drishti, a fictional livestreaming and short-video app, runs a seasonal campaign with a daily streak: watch or post on consecutive local days and unlock a reward on Monday. The product copy says "day" the way a user means it - midnight to midnight in Pacific time, where the company and most of its creators sit. The implementation means something else.

`APP_TIMEZONE` is `UTC`, which is correct for storage. Streak logic calls `now()→startOfDay()` and stores the result as the active date. At 11:30 p.m. Pacific on a Tuesday, that's already Wednesday UTC. A creator who posts once on Tuesday evening and once on Wednesday morning - two distinct Pacific calendar days - gets a single streak day, because both actions fall on the same UTC date. Support tickets pile up the week the campaign goes viral. Engineering "fixes" it by flipping the app timezone to `America/Vancouver`. Overnight, every `created_at` comparison, every TTL, and every cron that assumed UTC silently shifts by seven or eight hours depending on daylight saving, and Monday's payout job either double-fires or doesn't fire at all.

Timezone bugs are correctness bugs. They don't show up as 500s; they show up as the wrong calendar day, the wrong week boundary, a schedule that runs at the wrong wall-clock time, or an admin panel that displays yesterday as today. This chapter is about keeping two clocks straight - **UTC for instants**, a named **business timezone for calendar math** - and the edge cases that appear the moment a product says "day," "week," or "Monday at

midnight."

Instants and calendar days are different types

An *instant* is a point on the timeline: "this ticket was created at this exact moment," stored once, comparable everywhere. A *calendar day* is a civil label that depends on a timezone: "Tuesday, March 10" in Vancouver is not the same set of instants as "Tuesday, March 10" in UTC. Mixing them is the root failure mode.

Question	Use
When did this row get written?	UTC instant (<code>timestampTz</code> , <code>now()</code> in UTC)
Is this user still inside a 24-hour redeem window?	Duration from a UTC instant (<code>subHours(24)</code>)
Did they act on consecutive <i>local</i> days?	Convert to business TZ, then <code>toDatestring()</code>
Does "this week" mean Mon-Sun for the campaign?	Week boundaries in the business TZ
When should the Monday payout cron run?	Scheduler $\rightarrow$ <code>timezone(...)</code> matching that calendar

Laravel's `config('app.timezone')` should almost always be `UTC`. That makes `now()`, Eloquent timestamps, and queue `available_at` values mean the same thing on every server and in every region. Business calendar

semantics belong in application code as an explicit constant - not in `APP_TIMEZONE`, and not as a per-row column unless you genuinely need per-user timezones (wallets with local spending days, Chapter 21's `spendingDay`, are that case; a single-company campaign usually is not).

```
final class CampaignConfig
{
    /*
     * Calendar days, weeks, and payout schedules for this
     * product. DB timestamps stay UTC instants; convert to
     * this zone only when asking civil-date questions.
     */
    public const BUSINESS_TIMEZONE = 'America/Vancouver';
}
```

One constant beats a `timezone` column you will forget to set on half the rows, and beats three competing literals (`America/Vancouver`, `America/Los_Angeles`, `PST`) scattered across jobs.

Convert at the boundary, not in the middle

When an event arrives - a watch, a purchase, a ticket grant - keep the instant in UTC, and derive the civil day only where the product rule needs it:

```

public function recordActiveDay(
    int $userId,
    CarbonImmutable $activeAt,
): void {
    $activeDate = $activeAt
        →timezone(CampaignConfig::BUSINESS_TIMEZONE)
        →startOfDay();
    $today = $activeDate→toDateString();

    $progress = StreakProgress::query()
        →lockForUpdate()
        →firstOrCreate(
            [
                'user_id' => $userId,
                'active_date' => $today,
            ],
        );

    /*
     * ... streak increment using $today / yesterday
     * in the same business timezone
     */
}

```

The important move is `→timezone(BUSINESS_TIMEZONE)` before `startOfDay()` or `toDateString()`. Calling `startOfDay()` in UTC and then "displaying" Pacific is how you mint the wrong day. The inverse shows up in queries: when you need "everything since one week ago at Pacific midnight," compute that bound in the business zone and convert back to UTC for the SQL comparison:

```
$weekAgoPacificMidnightUtc = CarbonImmutable::now(
    CampaignConfig::BUSINESS_TIMEZONE,
)
    ->subWeek()
    ->startOfDay()
    ->utc();

Campaign::query()
    ->where('ends_at', '>=', $weekAgoPacificMidnightUtc)
    ->get();
```

That pattern - civil math in the business zone, storage and filters in UTC - is the same idea Chapter 21 uses when it freezes **spendingDay** at decision time so replay never re-asks "what is today?"

Schedulers must declare the timezone they mean

Laravel's scheduler inherits **app.timezone**. If that is UTC, then **->dailyAt('00:05')** means 00:05 UTC, not "just after midnight for the Pacific campaign." The failure mode from the opening incident is usually quieter than a wrong app timezone: one job uses **->timezone('America/Vancouver')**, a sibling job forgets and runs in UTC, and both claim to be "the Monday payout."

```

$chedule->job(new DisburseStreakRewards)
  ->weekly()
  ->mondays()
  ->at('00:05')
  ->timezone(CampaignConfig :: BUSINESS_TIMEZONE)
  ->withoutOverlapping()
  ->onOneServer();

$chedule->command('campaign:sync-all-items')
  ->dailyAt('06:00')
  ->timezone('UTC')
  ->withoutOverlapping()
  ->onOneServer();

```

Be explicit even when the answer is UTC. A codebase that mixes Vancouver payouts, New York compliance jobs, and Los Angeles digests is fine - as long as each entry names its zone next to the clock time, and that zone matches the product rule the job implements. A schedule entry with no `->timezone()` is not "timezone-agnostic"; it is "whatever `app.timezone` is today," which is the wrong kind of default for wall-clock work.

Uniqueness keys must use the same clock as the schedule

`ShouldBeUnique` and cache locks are timezone-sensitive when the key embeds a date or hour. A streak reward job that must run once per Pacific calendar day needs a Pacific day in `uniqueId()`. A missing-ticket generator that may safely run once per UTC hour needs a UTC hour. Using the wrong one either blocks a legitimate second run or allows a duplicate:

```
public function uniqueId(): string
{
    /*
     * Pacific calendar day - aligned with the Monday
     * Vancouver schedule, not with UTC midnight.
     */
    return $this->campaign->slug.' :: '.now(
        CampaignConfig::BUSINESS_TIMEZONE,
    )->format('Y-m-d');
}
```

```
public function uniqueId(): string
{
    // Hourly reconciliation: UTC hour is the right grain.
    return $this->campaign->slug.' :: '.now()
        ->format('Y-m-d-H');
}
```

If the unique key and the schedule disagree about which midnight matters, you will get exactly the incident the unique constraint was supposed to prevent - either skipped work or double payouts - and it will reproduce only near timezone boundaries.

"Day" sometimes means 24 hours

Not every product "day" is a calendar day. A redeem cap of "once per day" often means "no more than once in any rolling 24-hour window," which is a duration from a UTC instant:

```
$recent = Redeem::query()  
    ->where('user_id', $userId)  
    ->where('created_at', '>=', now()->subHours(24))  
    ->exists();
```

A weekly purchase limit that uses `startOfWeek()` in UTC while marketing promises "this Pacific week" is a quieter cousin of the streak bug. Decide deliberately:

- **Rolling duration** (`subHours(24)`, `subDays(7)`) when the rule is elapsed time.
- **Civil calendar** (business-TZ `startOfDay` / `startOfWeek`) when the rule is a named day or week the user would recognize on a wall calendar.

Write the choice down next to the constant. The incident is almost always someone using the other interpretation without noticing.

Don't write time as a magic number

Durations have the same readability problem as timezones: a bare `300` in a cache call does not say whether the author meant five minutes, and after Laravel started treating integer TTLs as seconds, half the codebase's "I thought that was minutes" assumptions became silent bugs. Prefer a helper that names the unit.

```
use function Illuminate\Support\{
    minutes,
    seconds,
    hours,
    days,
};

/*
 * Readable at the call site - five minutes, not "300 of
 * something." These return a CarbonInterval the cache
 * layer accepts directly.
 */
Cache::put('trending', $feed, minutes(5));
Cache::remember(
    'exchange-rate',
    seconds(30),
    fn () => $this->fetchRate(),
);
Cache::put('sitemap', $xml, hours(6));
Cache::put('annual-report', $pdf, days(1));
```

`now()->addMinutes(5)` (or `now()->plus(minutes: 5)`) is the same idea when you need an absolute expiry instead of an interval - Chapter 11's cache examples use that form. What to avoid is the unadorned integer and the arithmetic that pretends to document itself:

This is a sample from "Laravel After Deploy: Architecture, Performance, and Operations at Scale".